ADVANCED CALCULATOR API

TypeScript + Express 5 — Code Explanation & API Guide

A practical guide to understanding the project architecture, request lifecycle, validation, arithmetic engine, safe expression parser, chained calculations, history repository, and centralized API responses/errors.

Project component	What it teaches
API design	REST routes, HTTP methods, request/response contracts
TypeScript	Types, interfaces, strict validation and safer refactoring
Architecture	Routes → controllers → services → repository
Security	Input limits, finite-number checks, safe parsing, rate limiting
Operations	Environment config, request IDs, logging, testing

Prepared from the Advanced Calculator API project ZIP and the assignment requirements. This document explains the implementation and gives examples for running and testing it.

Contents

- 1. What the application does
- 2. Project structure and architecture
- 3. Installation and execution
- 4. HTTP routes and request examples
- 5. Centralized success response: ApiResponse
- 6. Centralized errors: ApiError and error middleware
- 7. Request IDs, logging, validation, and rate limiting
- 8. Arithmetic operation registry and numeric safety
- 9. Safe expression parser
- 10. Chained calculations and failed-step behavior
- 11. Calculation history and pagination
- 12. Configuration and environment variables
- 13. Automated tests and manual Postman testing
- 14. Exact API response contracts
- 15. Acceptance checklist
- 16. Important implementation notes and limitations
- 17. Quick revision / interview explanation

1. What the application does

The Advanced Calculator API is a backend service that accepts HTTP requests, validates inputs, performs calculations, and returns consistent JSON responses. It is not a frontend calculator; clients such as Postman, a browser-based UI, or another backend call its endpoints.

Core capabilities

- Six basic operations: add, subtract, multiply, divide, power, and modulo.
- Expression evaluation with +, -, *, /, %, parentheses, decimals, and unary negative values.
- Chained operations that run in the order supplied by the client.
- In-memory history with unique record IDs, timestamps, pagination, retrieval, and deletion.
- Request IDs, request logging, request validation, rate limiting, and centralized error formatting.
- A health endpoint at GET /health.

High-level request flow

```
Client / Postman
      |
      v
Request ID middleware  -> adds X-Request-ID
      |
      v
Request logger         -> records method, URL, status, duration
      |
      v
JSON body parser
      |
      v
Router + rate limiter + validation middleware
      |
      v
Controller             -> HTTP input/output only
      |
      v
Service                -> calculation/business rules
      |
      v
Repository             -> stores/reads history
      |
      v
ApiResponse            -> consistent success JSON

Errors from any layer -> not-found / error-handler -> consistent error JSON
```

> The request logger records the final HTTP status in the response finish event, so it can log failures as well as successes.

2. Project structure and architecture

Path	Responsibility
src/app.ts	Creates Express app, registers middleware, health route, routers, 404 and error handlers.
src/server.ts	Starts listening on configured port and handles SIGTERM/SIGINT shutdown.
src/config/env.ts	Loads .env and validates configuration at startup using Zod.
src/routes/*.routes.ts	Maps HTTP methods and paths to middleware/controllers.
src/controllers/*.controller.ts	Reads validated data, calls services, returns ApiResponse.
src/services/calculator.service.ts	Runs arithmetic/chains and saves successful results.
src/services/expression.service.ts	Tokenizes and parses mathematical expressions safely.
src/services/calculations.service.ts	Coordinates history reads/deletes.
src/repositories/calculations.repository.ts	Owns in-memory calculation history.
src/validators/*.validator.ts	Defines allowed body, query, and path-parameter shapes.
src/middlewares/*.ts	Request IDs, logs, validation, rate limiting, 404 and errors.
src/utils/api-response.ts	Standard success response envelope.
src/utils/api-error.ts	Structured operational errors with HTTP status and error code.
src/utils/calculate.ts	Operation registry and shared numeric safety checks.
tests/calculator.test.ts	Automated API tests using Vitest and Supertest.
postman_collection.json	Requests that can be imported into Postman.
.env.example / .gitignore	Sample settings and excludes local secrets/build output.

3. Installation and execution

Use Node.js 20 or later. Open a terminal in the extracted advanced-calculator-api folder.

Install dependencies

```
npm install
```

Create the local environment file

```
# Windows PowerShell
Copy-Item .env.example .env

# macOS / Linux
cp .env.example .env
```

Start the development server

```
npm run dev
```

Build and test

```
npm run build
npm test
```

When the server starts, its default base URL is http://localhost:3000. Visit GET http://localhost:3000/health or send that request from Postman.

Package scripts

Command	Meaning
npm run dev	Runs src/server.ts in watch mode using tsx.
npm run build	Compiles TypeScript to the dist directory.
npm start	Runs the compiled dist/server.js.
npm test	Runs the Vitest suite once.
npm run test:watch	Runs Vitest in watch mode.

4. HTTP routes and request examples

Base URL: http://localhost:3000

Calculator routes

Method	Path	Purpose
POST	/api/v1/calculator/add	Add a and b
POST	/api/v1/calculator/subtract	Subtract b from a
POST	/api/v1/calculator/multiply	Multiply a and b
POST	/api/v1/calculator/divide	Divide a by b
POST	/api/v1/calculator/power	Calculate a raised to b
POST	/api/v1/calculator/modulo	Remainder of a divided by b
POST	/api/v1/calculator/evaluate	Evaluate an expression string
POST	/api/v1/calculator/chain	Run a sequence of operations

Basic arithmetic request

```
POST /api/v1/calculator/add
Content-Type: application/json

{
  "a": 10,
  "b": 3
}
```

Expression request

```
POST /api/v1/calculator/evaluate
{
  "expression": "(10 + 5) * 2 - 8 / 4"
}
```

This expression evaluates to 28: parentheses first (15), then multiplication (30), division (2), and subtraction (28).

Chained calculation request

```
POST /api/v1/calculator/chain
{
  "initialValue": 100,
  "operations": [
    { "operator": "subtract", "value": 20 },
    { "operator": "multiply", "value": 2 },
    { "operator": "divide", "value": 4 }
  ]
}
```

Calculation history routes

Method	Path	Purpose
GET	/api/v1/calculations?page=1&limit;=10	List history with pagination
GET	/api/v1/calculations/:id	Get one record by UUID
DELETE	/api/v1/calculations/:id	Delete one record
DELETE	/api/v1/calculations	Clear all history
GET	/health	Health check

5. Centralized success response: ApiResponse

File: src/utils/api-response.ts

```
export class ApiResponse<T> {
  readonly success = true;

  constructor(
    readonly message: string,
    readonly data: T,
    readonly requestId: string
  ) {}

  static send<T>(
    res: Response,
    statusCode: number,
    message: string,
    data: T
  ) {
    return res.status(statusCode).json(
      new ApiResponse(
        message,
        data,
        res.locals.requestId as string
      )
    );
  }
}
```

The generic type T lets the response wrap different data shapes while keeping type information. The send() method sets the HTTP status and serializes one predictable JSON shape. Controllers call ApiResponse.send() instead of building their own success envelope.

Success contract

```
{
  "success": true,
  "message": "Calculation completed successfully.",
  "data": {
    "operation": "add",
    "inputs": { "a": 10, "b": 3 },
    "result": 13,
    "calculationId": "uuid",
    "createdAt": "ISO timestamp"
  },
  "requestId": "request-uuid"
}
```

6. Centralized errors: ApiError and error-handler

File: src/utils/api-error.ts

ApiError extends the built-in Error class and adds statusCode, code, details, and isOperational. Helper factories such as badRequest() and notFound() make common errors consistent. Services throw ApiError; they do not send HTTP responses directly.

Property	Purpose
statusCode	HTTP status to return, such as 400, 404, 429 or 500.
code	Machine-readable identifier such as DIVISION_BY_ZERO.
message	Readable explanation safe to return to the client.
details	Optional structured context, for example validation issues or failed chain step.
isOperational	Marks the error as an expected application error.

File: src/middlewares/error-handler.ts

The error handler is registered last. It recognizes ApiError, malformed JSON parser errors, and oversized request bodies. Unknown exceptions become a generic INTERNAL_SERVER_ERROR. Stack traces are not sent to the client; server logs may include the stack in non-production mode.

Error contract

```
{
  "success": false,
  "error": {
    "code": "DIVISION_BY_ZERO",
    "message": "Division by zero is not allowed.",
    "details": null
  },
  "requestId": "request-uuid"
}
```

7. Request IDs, logging, validation, rate limiting

Request ID — src/middlewares/request-id.ts

randomUUID() creates an ID for each incoming request. It is stored in res.locals.requestId and returned as the X-Request-ID header. The same ID appears in success and error bodies and logs, making a request traceable across the lifecycle.

Request logger — src/middlewares/request-logger.ts

It records the HTTP method, original URL, final status code, elapsed time in milliseconds, and request ID. It deliberately avoids logging request bodies.

Validation — validators and validate-request.ts

Zod schemas define the allowed data. The middleware calls safeParse(). Invalid requests are sent to the centralized error handler before controllers run. Valid parsed data is stored in res.locals.validatedBody, validatedQuery, or validatedParams.

Rate limiter — src/middlewares/rate-limiter.ts

express-rate-limit limits requests to the calculator router by client IP. RATE_LIMIT_WINDOW_MS and RATE_LIMIT_MAX_REQUESTS are configured in .env. When the limit is exceeded, it returns HTTP 429 with the same error envelope.

8. Arithmetic registry and numeric safety

File: src/utils/calculate.ts

The operationRegistry maps an operation name to a function. A shared calculate() function calls the selected operation and checks that the result is finite and within MAX_ABSOLUTE_VALUE. This avoids duplicating the safety checks in every controller.

Operation	Rule
add	a + b
subtract	a - b
multiply	a * b
divide	Reject b = 0, otherwise a / b
power	Reject exponents beyond MAX_EXPONENT; guard 0 to a negative exponent
modulo	Reject b = 0, otherwise a % b

Inputs are validated as actual JSON numbers; strings such as "10" are not silently converted into numbers. Non-finite values and out-of-range results are rejected.

9. Safe expression parser

File: src/services/expression.service.ts

The parser does not use eval() or execute JavaScript. It converts the expression into tokens and then parses those tokens according to grammar rules.

Parser part	What it does
tokenize()	Reads numbers, operators, and parentheses; rejects unsupported characters.
expression()	Handles addition and subtraction after parsing terms.
term()	Handles multiplication, division, and modulo.
unary()	Handles unary + and - signs.
primary()	Reads a numeric literal or a parenthesized expression.
parse()	Ensures every token was consumed; leftover tokens mean malformed input.

Precedence follows the call hierarchy: expression → term → unary → primary. Since term() is evaluated before expression() combines + and -, multiplication/division take precedence over addition/subtraction. Parentheses recursively call expression().

Supported examples

```
"10 + 5 * 2"       -> 20
"(10 + 5) * 2"     -> 30
"-5 + 2.5"         -> -2.5
"8 / 0"            -> DIVISION_BY_ZERO
"10 + process..."  -> INVALID_EXPRESSION
```

The expression syntax intentionally supports +, -, *, /, %, parentheses, decimal values and negative numbers. Exponentiation is available through the /power endpoint, not through an expression operator.

10. Chained calculations and failed steps

File: src/services/calculator.service.ts

performChain() starts with initialValue, loops through operations in array order, and uses each result as the next operation's input. Each successful step is appended to steps. If calculate() throws ApiError, the service wraps it with a message such as “Chain failed at step 3” and details describing the operation and operands.

Example

```
initialValue = 100
subtract 20  -> 80
multiply 2   -> 160
divide 4     -> 40
final result -> 40
```

The Zod chain schema validates the initial value, requires at least one operation, caps the number of operations, restricts operators to an enum, validates each operand, and rejects unexpected fields before the controller calls the service. Runtime failures stop execution at the first failing step. Only a fully successful chain is saved in history.

11. Calculation history and pagination

Repository: src/repositories/calculations.repository.ts

A Map stores records by UUID. Each record contains id, operation, inputs, result, and createdAt. The repository offers create(), findAll(), findById(), deleteById(), and clear().

Service: src/services/calculations.service.ts

The service coordinates repository operations and throws CALCULATION_NOT_FOUND if a requested record does not exist. The controller converts successful results to ApiResponse.

Pagination

```
GET /api/v1/calculations?page=1&limit=10
```

The repository calculates start = (page - 1) × limit, returns that slice, and includes page, limit, total, and totalPages. The query validator accepts positive integer strings and supplies defaults page=1 and limit=10.

In-memory means data exists only while this Node.js process is running. Restarting the server clears history. A persistent database would be a future extension, not part of this assignment's in-memory requirement.

12. Configuration and environment variables

File: src/config/env.ts

dotenv loads .env. Zod checks environment values at startup. If the configuration is invalid, the process logs the validation failure and exits instead of starting with unsafe settings. env is exported as a frozen object for the rest of the application.

Variable	Default	Meaning
PORT	3000	HTTP server port
NODE_ENV	development	Environment mode
RATE_LIMIT_WINDOW_MS	60000	Rate-limit window in milliseconds
RATE_LIMIT_MAX_REQUESTS	30	Maximum requests per client in the window
MAX_EXPRESSION_LENGTH	200	Maximum expression length
MAX_CHAIN_OPERATIONS	50	Maximum number of chain steps
MAX_ABSOLUTE_VALUE	1e12	Absolute bound for numbers/results
MAX_EXPONENT	100	Absolute exponent bound
MAX_BODY_SIZE	16kb	Maximum JSON request body size

13. Automated tests and manual Postman testing

The project uses Vitest as the test runner and Supertest to make HTTP requests against the Express app without needing to open a real network port. The tests cover arithmetic, validation, expression parsing, chain behavior, history, request IDs, and route/error behavior.

Run automated checks

```
npm test
npm run build
```

Postman workflow

- Import postman_collection.json into Postman.
- Use http://localhost:3000 as the base URL.
- Send GET /health first.
- Try POST /api/v1/calculator/add with {"a":10,"b":3}.
- Try POST /api/v1/calculator/evaluate with {"expression":"(10 + 5) * 2 - 8 / 4"}.
- Try a chain that divides by zero at step 2 or 3 and inspect error.details.failedStep.
- Create a successful calculation, list history, copy its id, fetch it, delete it, and verify it is no longer present.
- Send invalid types, missing fields, unknown paths, and repeated requests to test error and rate-limit behavior.

14. Exact API response contracts

Success response

```
{
  "success": true,
  "message": "Human-readable success message.",
  "data": { "...": "endpoint-specific data" },
  "requestId": "UUID"
}
```

Error response

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request body validation failed.",
    "details": [
      { "path": "a", "message": "Required" }
    ]
  },
  "requestId": "UUID"
}
```

Typical status codes

Status	When used
200 OK	Successful calculation, history read, or deletion
400 Bad Request	Invalid body/query/params, malformed JSON, invalid expression, divide-by-zero
404 Not Found	Unknown route or missing calculation record
413 Payload Too Large	Request body exceeds configured limit
429 Too Many Requests	Calculation rate limit exceeded
500 Internal Server Error	Unexpected server-side exception

15. Explicit acceptance checklist

Check	How to verify
[ ] All six arithmetic operations	Send a POST request to each basic operation route.
[ ] Decimals and negative numbers	Try a=-5.5 and b=2.25.
[ ] Wrong types/missing fields	Send strings, null, or omit a/b.
[ ] Divide/modulo by zero	Set b=0 and check error code/message.
[ ] Expression precedence	Try 10 + 5 * 2 and (10 + 5) * 2.
[ ] Malformed/long expression	Try 2 + * 3 and exceed MAX_EXPRESSION_LENGTH.
[ ] Entire chain validated first	Include an invalid operator or malformed step; ensure validation rejects before service execution.
[ ] Chain stops on runtime failure	Place divide by zero in step 2 and check failedStep=2.
[ ] History create/read/delete	Calculate, list, get by ID, delete, then verify 404.
[ ] Pagination	Use page=1&limit;=2 and inspect metadata.
[ ] Request ID	Check X-Request-ID header and matching response requestId.
[ ] Logging	Inspect console for method, URL, status, duration, request ID.
[ ] Rate limiting	Exceed configured request count within the time window; expect 429.
[ ] Unknown route / unexpected error	Call a nonexistent URL and run the automated test suite.
[ ] Configuration and docs	Check .env.example, .gitignore, README, and Postman collection.

16. Important implementation notes and limitations

The guide describes the project as present in the supplied ZIP. Before submitting, run the build and tests locally; a ZIP existing on disk does not by itself prove that the project compiles or that every test passes.

Things to verify carefully

- The API's expression parser accepts +, -, *, /, %, parentheses and unary signs; it does not implement a ^ exponent operator. Use POST /api/v1/calculator/power for exponentiation.
- The rate limiter is mounted on the calculator router, so history endpoints are not rate-limited by that middleware.
- The history repository uses process memory. Restarting the service clears all saved records.
- Request body/query/path validation results are stored in res.locals to avoid mutating Express 5's getter-based req.query property.
- A failed arithmetic calculation is not stored as a successful history record. The chain saves one history record only after all steps succeed.
- The rate-limit response is formatted to match the shared error contract even though it is produced by the limiter middleware rather than the general error handler.

Potential improvement

For larger production applications, you could add structured logging, a database repository, OpenAPI/Swagger documentation, decimal precision policy, and more explicit tests for request validation and rate limiting. These are extensions; the current assignment specifically requests an in-memory repository.

17. Quick revision / interview explanation

One-minute explanation

“I built an Advanced Calculator REST API using TypeScript and Express 5. I separated routes, controllers, services, validators, middleware, and a repository so each layer has one responsibility. Zod validates requests before controllers run. Arithmetic operations are registered in one reusable operation registry, while mathematical expressions are evaluated with a safe tokenizer and recursive-descent parser instead of eval(). Chained calculations execute in order and stop on the first runtime failure, reporting the failed step. Every successful response uses ApiResponse, while ApiError and centralized error middleware standardize failures. Request IDs, request logging, rate limiting, environment validation, pagination, and Vitest/Supertest tests make the service easier to operate and maintain.”

Common questions

Question	Answer
Why TypeScript?	It adds static types that catch many mistakes during development and improves maintainability.
Why separate controllers and services?	Controllers handle HTTP; services contain business logic. This keeps logic testable and reusable.
Why ApiResponse?	It ensures every successful endpoint returns the same outer JSON structure.
Why ApiError?	It carries HTTP status, stable error code, safe message, and optional details to the error middleware.
Why not eval()?	eval can execute arbitrary JavaScript. A tokenizer/parser accepts only the defined mathematical grammar.
Why a repository layer?	It isolates history storage so the implementation can later switch from Map to a database.
How does request tracing work?	A UUID is generated once, returned in X-Request-ID, and reused in logs and response bodies.
What happens if a chain step fails?	The loop stops immediately and returns a structured error naming the failed step; no final successful chain record is saved.
What are the limitations?	History is in memory and expression syntax excludes exponentiation; build/tests must be run in the target environment.

End of guide — Advanced Calculator API (TypeScript + Express 5)